



Projekt

Webová aplikace pro správu chytrého domu

Studijní program:

N2612 – Elektrotechnika a informatika

Studijní obor:

1802T007 – Informační technologie

Autor práce:

Martin Těhník

Vedoucí práce:

Ing. Petr Bílek, Ph.D.

MTI

Liberec 2026

KONCEPT ZADÁNÍ BAKALÁŘSKÉHO PROJEKTU

Jméno a příjmení: **Martin Těhník**
Název práce: **Webová aplikace pro správu chytrého domu**
Zadávací katedra: **Ústav mechatroniky a technické informatiky**
Vedoucí práce: **Ing. Petr Bílek Ph.D.**
Rozsah práce: **15–20 stran**

V Liberci dne

.....
Ing. Petr Bílek Ph.D.

Prohlášení

Prohlašuji, že svou bakalářskou práci jsem vypracoval/a samostatně jako původní dílo s použitím uvedené literatury a na základě konzultací s vedoucím mé bakalářské práce a konzultantem.

Jsem si vědom/a toho, že na mou bakalářskou práci se plně vztahuje zákon č. 121/2000 Sb., o právu autorském, zejména § 60 – školní dílo.

Beru na vědomí, že Technická univerzita v Liberci nezasahuje do mých autorských práv užitím mé bakalářské práce pro vnitřní potřebu Technické univerzity v Liberci.

Užiji-li bakalářskou práci nebo poskytnu-li licenci k jejímu využití, jsem si vědom/a povinnosti informovat o této skutečnosti Technickou univerzitu v Liberci; v tomto případě má Technická univerzita v Liberci právo ode mne požadovat úhradu nákladů, které vynaložila na vytvoření díla, až do jejich skutečné výše.

Současně čestně prohlašuji, že text elektronické podoby práce vložený do IS STAG se shoduje s textem tištěné podoby práce.

Beru na vědomí, že má bakalářská práce bude zveřejněna Technickou univerzitou v Liberci v souladu s § 47b zákona č. 111/1998 Sb., o vysokých školách a o změně a doplnění dalších zákonů (zákon o vysokých školách), ve znění pozdějších předpisů.

Jsem si vědom/a následků, které podle zákona o vysokých školách mohou vyplývat z porušení tohoto prohlášení.

{Datum prohlášení}

Martin Těhník

TECHNICKÁ UNIVERZITA V LIBERCI

Fakulta mechatroniky, informatiky a mezioborových studií

Ústav MTI

Školní rok: 2025/2026

ZADÁNÍ ROČNÍKOVÉHO PROJEKTU

Název projektu: Webová aplikace pro správu chytrého domu

Řešitel(-é): Martin Těhník

Studijní obor: B0613A140005AI / Aplikovaná informatika

Vedoucí projektu: Ing. Petr Bílek, Ph.D.

Zadání:

- 1) Vypracujte rozvahu možných řešení, přístupů a frameworků pro tvorbu webových aplikací.
- 2) Vyberte nejvhodnější řešení s ohledem na cíl této práce.
- 3) Navrhněte blokové schéma propojení příslušných komponent.
- 4) Vytvořte vývojový diagram aplikace a realizujte aplikaci na příslušném hardware.
- 5) Vytvořte databázi a statisticky zpracujte a vizualizujte uložená data.
- 6) Přidejte možnost ovládání výstupů a nastavení parametrů senzorů.
- 7) Otestujte aplikaci a zhodnoťte dosažené parametry.

Doporučená literatura:

- [1] Maciej Kranz: Building the Internet of things; ISBN-13: 978-1119285663; Wiley
[2] Steven Goodwin: Smart Home Automation with Linux; ISBN-13: 978-1430227786;

Požadované náležitosti:

Projekt PRJ1: K zápočtu v zimním semestru předložit laboratorní deník a stručnou průběžnou zprávu o dosažených výsledcích.

Projekt PRJ2: K obhajobě předložit zprávu o řešení projektu (15 stran, písmo 12 b) včetně anotace v českém a anglickém jazyce (viz [1]). Prezentace výsledků (max. 15 minut), ukázky realizovaného díla atp.

Termíny:

Odevzdání dokumentace a obhajoba podle harmonogramu výuky na FM.

Datum zadání: 8.10.2025

Podpis vedoucího projektu:

Webová aplikace pro správu chytrého domu

Abstrakt

Cílem této práce je vytvořit webovou aplikaci pro správu chytrého domu, která bude provozována na jednodeskovém počítači s Linuxem. Aplikace bude umožňovat sběr a ukládání dat ze simulovaných senzorů, jejich vyhodnocení a vizualizaci. Součástí řešení je také ovládání výstupů a možnost nastavení parametrů jednotlivých senzorů. Při realizaci budou porovnány dostupné technologie pro vývoj webových aplikací a vybrán vhodný framework, například Django nebo Flask. Výsledná aplikace bude otestována na zvoleném hardware a budou zhodnoceny její dosažené parametry.

Klíčová slova: Python, SQLite, Flask, Smarthome

Web application for smarthome control

Abstract

The aim of this thesis is to create a web application for smart home management that will run on a single-board computer with Linux. The application will enable the collection and storage of data from simulated sensors, as well as its evaluation and visualization. The solution also includes output control and the ability to set parameters for individual sensors. During implementation, available technologies for web application development will be compared and a suitable framework, such as Django or Flask, will be selected. The resulting application will be tested on the selected hardware and its achieved parameters will be evaluated.

Keywords: Python, SQLite, Flask, Smarthome

Poděkování

Rád bych poděkoval všem, kteří přispěli ke vzniku tohoto díla.

Obsah:

1. Teoretická část	9
1.1 Smart Home	9
1.1.1 Konkurence v Smart Home.....	9
1.1.2 Trendy v Smart Home.....	9
1.2 Jednodeskové počítače.....	10
1.2.1 Radxa	10
1.2.2 Raspberry	10
1.2.3 ESP32.....	11
1.2.4 M5stack.....	11
1.3 Webový framework.....	12
1.4 Flask.....	12
1.5 HTTP protokol	13
1.7 Tailscale VPN.....	14
2. Teoretická část.....	15
2.1 Rozvaha možných řešení – Frameworky	15
2.1.1 Čistý přístup bez frameworku	15
2.1.2 Node.js – Express.js.....	15
2.1.3 Node.js – Next.js.....	15
2.1.4 Java – Spring Boot	15
2.1.5 Go – Gin.....	16
2.1.6 Python – Django	16
2.1.7 Python – FastAPI	16
2.1.7 Zvolené řešení – Python – Flask.....	16
2.2 Rozvaha možných řešení – Hardware.....	17
2.3 Návrh.....	17
2.4 Realizace aplikace.....	18
2.4.1 Zabezpečení aplikace	18
2.5 Funkcionalita.....	19
2.5.1 Sběr a ukládání dat ze senzorů.....	19
2.5.2 Webový dashboard a widgety	20
2.5.3 Historická data a grafické zobrazení.....	20
2.5.4 Ovládání relé a fyzických spotřebičů.....	20
2.5.5 Nastavení systému	20
2.5.6 Sít'ová infrastruktura	21

2.6 Testování	21
2.6.1 Automatizované testování	21
2.7.1 Manuální testování	23
Závěr	25
Bibliografie	26
Přílohy	27
A Příloha	27

Seznam obrázků

Obrázek 1: Schéma webové stránky	12
Obrázek 2: Schéma HTTP protokolu	13
Obrázek 3: Návrh projektu	17
Obrázek 4: Ukázka UI	18
Obrázek 5: Návrh aplikace	19
Obrázek 6: Ukázka testování	23

Seznam zkratek

ARM	Advanced RISC Machine
IoT	Internet of Things
HTTP	Hypertext Transfer Protocol
HTML	Hypertext Markup Language
API	Application Programming Interface
SQL	Structured Query Language
VPN	Virtual Private Network
Wi-Fi	Wireless Fidelity
MQTT	Message Queuing Telemetry Transport
GPIO	General Purpose Input/Output
CPU	Central Processing Unit
OS	Operating System
SBC	Single Board Computer
URL	Uniform Resource Locator
REST	Representational State Transfer

Úvod

Téma SmartHome zažívá v posledních letech velký rozvoj, a to především díky široké škále možností, které nabízí. V dnešní uspěchané době lidé stále hledají jakýkoliv způsob úspory času a peněz, čehož nám pomůže docílit chytrá domácnost. Díky optimalizaci využití elektrické energie ze solárních panelů či distribuční sítě lze snížit náklady na elektřinu a zároveň efektivněji využít dostupné zdroje. Chytré řízení domu obecně přispěje k lepší životní úrovni zákazníka, jelikož se nebude muset starat o manuální nastavení jednotlivých prvků domu. Nastaví je pouze při prvotní instalaci a pak už budou pracovat na základě přednastavených kritérií. SmartHome bude tak čím dál více využíván běžnou populací ať už díky klesající ceně nebo díky rostoucí uživatelské přívětivosti. Tato práce se zaměřuje na vývoj systému chytré domácnosti, který bude postaven na jednodeskovém počítači s Linuxovým operačním systémem a umožní prostřednictvím webové aplikace sbírat, ukládat a vyhodnocovat data ze senzorů, vizualizovat je a současně ovládat připojená zařízení a nastavovat parametry zařízení.

1. Teoretická část

1.1 Smart Home

Smart Home představuje moderní koncept propojení technologií a domácího prostředí s cílem zvýšit komfort, bezpečnost a energetickou efektivitu. Základem chytré domácnosti je síť propojených zařízení, která lze ovládat na dálku pomocí mobilní aplikace, hlasových asistentů nebo automatizovaných scénářů. Mezi běžné prvky patří například chytré osvětlení, termostaty, bezpečnostní kamery, senzory pohybu, chytré zámky či domácí spotřebiče. Tato zařízení spolu komunikují prostřednictvím internetu a dalších jiných bezdrátových technologií (tzv. Internet of Things – IoT). Smart Home systémy přinášejí nejen pohodlí a úsporu času, ale také mohou významně snížit spotřebu energie a zvýšit bezpečnost domácnosti díky automatizaci a vzdálenému monitorování.

1.1.1 Konkurence v Smart Home

Na trhu v současnosti existuje několik konkurenčních platforem a ekosystémů pro správu chytré domácnosti. Mezi nejrozšířenější komerční řešení patří Apple HomeKit, Google Home a Amazon Alexa, které nabízejí uživatelsky přívětivé prostředí, avšak vyžadují závislost na cloudových službách výrobce. Průmyslovým standardem pro profesionální instalace je protokol KNX, který se vyznačuje vysokou spolehlivostí a škálovatelností, ale také vyšší pořizovací cenou. Pro pokročilé uživatele a vývojáře je populární open-source platforma Home Assistant, která umožňuje lokální provoz bez závislosti na internetu a podporuje tisíce různých zařízení. Každý z těchto ekosystémů přináší odlišný přístup k automatizaci, zabezpečení a integraci zařízení. Tato práce se zaměřuje na vlastní implementaci lokálního systému inspirovaného principy Home Assistantu, který nevyžaduje cloudové služby a je plně pod kontrolou uživatele.

1.1.2 Trendy v Smart Home

V roce 2026 se trendy ve **smart home** přesouvají od pouhého ovládání přes mobil k plné autonomii a hluboké integraci. Klíčovým tématem je **prediktivní automatizace poháněná AI**, kdy dům již nečeká na váš povel, ale díky pokročilým senzorům přítomnosti a strojovému učení sám předvídá vaše potřeby – od úpravy osvětlení podle biorytmu až po optimalizaci vytápění v reakci na ceny energií a předpověď počasí.

Standard **Matter** se stal definitivní normou, což konečně vyřešilo dřívější roztříštěnost trhu a umožňuje bezproblémovou komunikaci mezi zařízeními různých značek (Apple, Google, Samsung či Amazon) v rámci jednoho ekosystému. Velký důraz je kladen také na:

- **Energetickou soběstačnost:** Integrace solárních panelů a domácích baterií přímo do chytrého řízení domu.
- **Pokročilé zabezpečení:** Využití biometrie (např. rozpoznávání žil v dlani u zámků) a lokálního zpracování dat z kamer pro maximální ochranu soukromí.
- **Domácí robotiku:** Příchod multifunkčních robotů, kteří zvládnou více než jen vysávání, například pomoc při vaření nebo asistenci starším osobám.

1.2 Jednodeskové počítače

Dnes jsou na trhu různé varianty jednodeskových počítačů od méně výkonných a cenově dostupných jako například ESP32 až po Raspberry Pi, které je schopné nahradit desktop pro základní použití. Ve světě IoT se používají ve velké míře Raspberry Pi a Orange Pi díky jejich bohatému I/O a výkonu. Mezi zařízeními s větším výkonem už nebývá tolik rozdílů, které by hovořili pro jednu či druhou variantu, proto se můžeme orientovat situací na trhu a dané ceně za kus. Nakonec jsme se rozhodli pro Raspberry Pi, jelikož Radxa byla mimo finanční předpoklady, kvůli své zabudované flash paměti.

- **Raspberry Pi** – nejrozšířenější platforma pro domácí automatizaci. Nabízí širokou podporu Linuxových distribucí, velkou komunitu, dostupnost příslušenství a spolehlivý provoz.
- **Orange Pi** – alternativa k Raspberry Pi s podobnou koncepcí. Nabízí vyšší výkon u některých modelů, často nižší cenu, ale menší komunitní podporu a občas složitější ovladače.
- **Radxa Rock Pi** – výkonné SBC, u některých modelů možnost připojení SSD přes M.2, lepší hardware, ale méně rozsáhlá komunita a složitější dostupnost.
- **Arduino** – mikrokontrolerová platforma, nevhodná jako hlavní server (nemá OS Linux), vhodná pro řízení jednoduchých periférií a senzorů.
- **ESP32** – mikrokontroler s Wi-Fi a Bluetooth, ideální pro senzory a IoT zařízení. Nelze použít jako hlavní aplikační server, spíše doplněk k centrální jednotce.

1.2.1 Radxa

Společnost Radxa je technologická firma zaměřená na vývoj a výrobu embedded počítačů a vývojových desek určených pro výuku, výzkum, průmyslové aplikace i domácí projekty. Byla založena v Číně a proslavila se zejména svou řadou Rock Pi, která nabízí výkonné alternativy k populárním deskám jako je Raspberry Pi. Radxa využívá moderní procesory ARM (např. Rockchip, Amlogic či NXP) a nabízí širokou škálu zařízení – od miniaturních modelů Radxa Zero až po výkonné desky Rock 5 s podporou rozhraní NVMe, USB-C, PCIe a gigabitového Ethernetu. Firma klade důraz na otevřený ekosystém, podporu Linuxu, Androidu i Debianu a na aktivní komunitu vývojářů, kteří její produkty využívají v oblasti IoT, robotiky, edge computingu i multimediálních aplikací.

1.2.2 Raspberry

Raspberry Pi představuje klíčový prvek v oblasti vestavěných systémů a edukačních platforem, který v podstatě demokratizoval přístup k vývoji hardwaru a nízkoúrovňovému programování. Z pohledu studenta technického oboru je tato jednodesková platforma neocenitelná především díky své flexibilitě, kdy na bázi architektury ARM umožňuje provozovat plnohodnotné distribuce Linuxu při zachování minimální energetické náročnosti. Klíčovou komparativní výhodou je přítomnost GPIO rozhraní, které slouží jako most mezi softwarovým prostředím a fyzickým světem, což z Raspberry dělá ideální uzel pro sběr dat ze senzorů v rámci IoT sítí nebo řídicí jednotku pro mechatronické systémy. Ať už jde o rychlé prototypování v Pythonu, stavbu vlastního domácího clusteru pro distribuované výpočty, nebo nasazení v roli lokálního serveru s Dockerem, Raspberry Pi zůstává nepřekonaným standardem.

1.2.3 ESP32

Zatímco výkonné jednodeskové počítače přebírají v rámci hierarchie inteligentních budov roli centrálních serverů a bran, pro samotný sběr dat z fyzického prostředí a přímou interakci s periferiemi se využívají specializované mikrokontrolery. V současném světě internetu věcí zaujímá dominantní postavení čipová sada ESP32 od společnosti Espressif Systems. Jedná se o vysoce integrovaný systém na čipu, který v rámci jediného pouzdra kombinuje výpočetní jednotku s nízkoenergetickými bezdrátovými technologiemi Wi-Fi a Bluetooth.

Implementace platformy ESP32 do projektů chytré domácnosti je podložena několika klíčovými technickými parametry:

- Energetická efektivita: Čip disponuje pokročilým managementem napájení, včetně režimu deep sleep, v němž spotřeba klesá do řádů jednotek mikroampér. Tato vlastnost je důležitá pro nasazení samostatných uzlů napájených z bateriových článků, u nichž lze dosáhnout provozní životnosti v řádu měsíců až let.
- Výpočetní výkon a bezpečnost: Architektura založená na dvoujádrovém procesoru s nastavitelnou taktovací frekvencí až 240 MHz poskytuje dostatečnou rezervu pro realizaci asymetrického šifrování (např. TLS/SSL) a lokální předzpracování signálu (edge computing), což snižuje datovou zátěž přenášenou na centrální server.
- Periferní konektivita a rozšiřitelnost: Integrovaná paleta hardwarových rozhraní, zahrnující standardy I2C, SPI, UART a vícekanálové ADC/DAC převodníky (většinou jako příslušenství), umožňuje bezprostřední integraci širokého spektra senzorických prvků pro monitoring environmentálních veličin (teplota, relativní vlhkost, koncentrace CO₂) i ovládání výkonových akčních členů.

1.2.4 M5stack

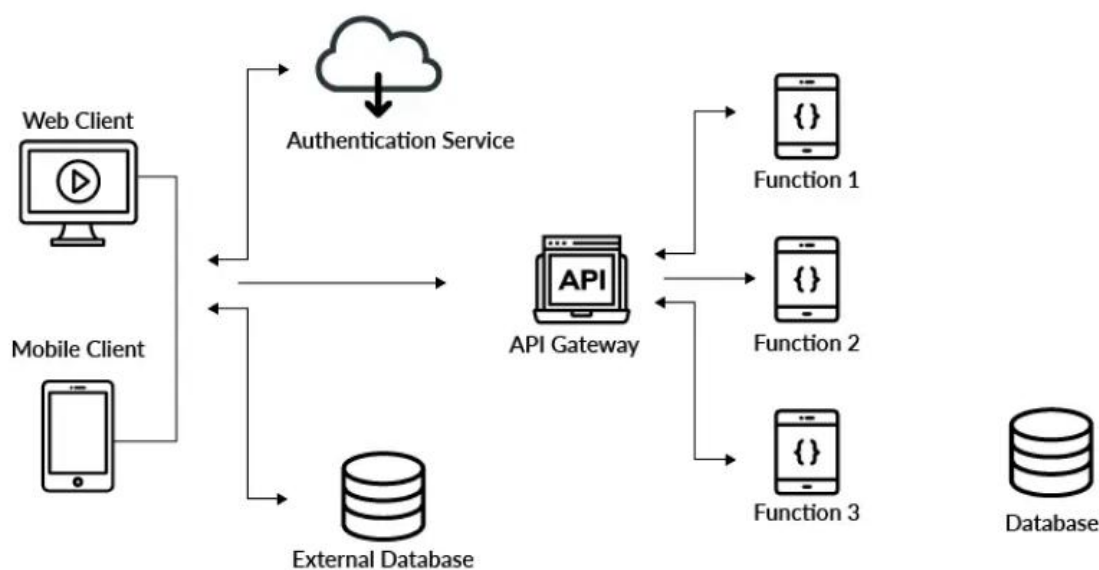
Platforma M5Stack představuje pokročilý ekosystém založený na čipu ESP32, který integruje mikrokontroler do kompaktního průmyslového pouzdra. Na rozdíl od běžných vývojových desek nabízí hotové řešení s vestavěným LCD displejem, ovládacími tlačítky a správou napájení, což eliminuje potřebu návrhu vlastních mechanických krytů a doplňkových obvodů.

Klíčovou vlastností je vysoká modularita založená na stohování modulů přes sběrnici M-Bus. Tímto způsobem lze systém snadno rozšířit o specifické funkce, jako jsou reléové výstupy, senzory prostředí nebo různé komunikační protokoly (LoRa, RS485), bez nutnosti pájení.

V architektuře chytrého domu plní M5Stack roli tzv. edge device (koncového zařízení). Slouží jako inteligentní uzel, který zajišťuje lokální sběr dat, jejich předběžné zpracování a následnou komunikaci s centrálním serverem (v tomto případě Radxa). Tato platforma je v rámci IoT projektů referenčním příkladem metodiky Rapid Prototyping, která výrazně zkracuje přechod od návrhu k nasazení funkčního zařízení v interiéru.

1.3 Webový framework

Webový framework je softwarová platforma, která usnadňuje a urychluje vývoj webových aplikací. Poskytuje strukturu a sadu nástrojů, knihoven a pravidel, které pomáhají vývojářům vytvářet kvalitní a udržitelné. Frameworky obvykle zajišťují základní funkcionality, jako je směrování (routing), práce s databází, autentizace uživatelů nebo například cachování dat. Tyto frameworky otevřeli cestu k rychlejšímu, a především snadnějšímu vývoji webových aplikací. V posledních letech se rozmohl vývoj JavaScriptu a jeho frameworků, které se dnes používají v mnoha aplikacích. Mezi nejznámější patří React nebo Vue.js. Nakonec jsem použil framework Flask v jazyce Python. Na obrázku je vyobrazena typická architektura webového frameworku. **Obrázek 1**



Obrázek 1: Schéma webové stránky

1.4 Flask

Flask je populární mikro-framework pro vývoj webových aplikací v jazyce Python, který sází na jednoduchost a flexibilitu. Na rozdíl od robustnějších nástrojů (jako je Django) nenutí vývojáře používat konkrétní strukturu složek nebo konkrétní databáze, což z něj dělá ideální volbu pro menší projekty, mikroslužby nebo rychlé prototypování. Flask poskytuje pouze nezbytné jádro pro směrování (routing) a zpracování HTTP požadavků, zatímco vše ostatní – od autentizace až po práci s databázemi – lze snadno přidat pomocí široké škály rozšíření. Díky své „minimalistické“ povaze a snadné čitelnosti kódu je Flask nesmírně oblíbený mezi začátečníky, kteří se chtějí rychle naučit základy webového vývoje, i mezi profesionály, kteří vyžadují plnou kontrolu nad každou komponentou své aplikace.

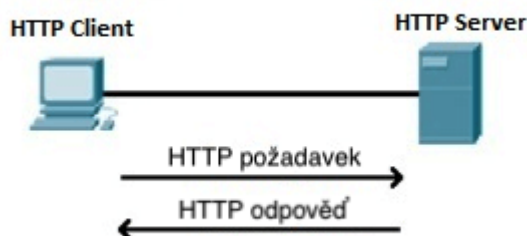
1.5 HTTP protokol

Moderní systémy automatizace a řízení budov dnes prioritně využívají architekturu klient-server, která tvoří robustní základ pro interakci mezi koncovým uživatelem a řídicím uzlem. Tato komunikace je realizována prostřednictvím protokolu HTTP (*Hypertext Transfer Protocol*), jenž funguje na principu request-response cyklu. V tomto modelu vystupuje klientské zařízení (např. webový prohlížeč) jako iniciátor požadavku, který je následně zpracován serverovou částí systému. Server požadavek validuje, provede příslušnou logickou operaci a vrátí strukturovanou odpověď obsahující stavový kód a případná data.

Z hlediska funkčnosti jsou pro ovládání technických zařízení klíčové zejména dvě HTTP metody:

- GET: Semanticky určená pro bezpečné a idempotentní získávání informací (např. dotaz na aktuální teplotu v místnosti nebo stav osvětlení), aniž by docházelo k modifikaci stavu systému.
- POST: Využívaná k přenosu dat, která vyvolávají změnu na straně serveru (např. odeslání příkazu k sepnutí akčního členu nebo aktualizaci konfiguračních parametrů).

Zásadním aspektem protokolu HTTP je jeho bezstavovost (*statelessness*). Tato vlastnost implikuje, že server neuchovává kontext mezi jednotlivými transakcemi, což vede k vyšší odolnosti systému vůči výpadkům spojení a usnadňuje škálovatelnost. Volba webového rozhraní jako primárního kontrolního bodu je strategickým rozhodnutím, které zajišťuje vysokou míru interoperability. Díky využití standardizovaných internetových protokolů se systém stává nezávislým na konkrétní platformě či operačním systému, čímž odpadá nutnost vývoje nativních aplikací a uživateli je umožněn přístup z libovolného terminálu vybaveného standardním prohlížečem. Na obrázku je znázorněna standardní komunikace pomocí HTTP requestů. **Obrázek 2**



Obrázek 2: Schéma HTTP protokolu

1.7 Tailscale VPN

Tailscale je moderní síťová služba založená na protokolu **WireGuard**, která umožňuje vytvořit zabezpečenou privátní síť (tzv. mesh VPN) mezi vašimi zařízeními bez ohledu na to, kde se nacházejí. Na rozdíl od tradičních VPN nevyžaduje Tailscale složité nastavování firewallů ani veřejné IP adresy, protože využívá technologii NAT traversal k přímému propojení koncových bodů. Každé zařízení v síti získá svou stabilní interní IP adresu a komunikace probíhá skrze plně šifrované „tunely“, což zajišťuje maximální soukromí a rychlost. Služba je mimořádně oblíbená pro svou jednoduchost, kdy se stačí přihlásit přes existujícího poskytovatele identity (např. Google nebo Microsoft), a během okamžiku můžete bezpečně přistupovat ke svému domácímu NAS serveru, pracovní stanici nebo vývojovému prostředí odkudkoliv na světě.

2. Teoretická část

2.1 Rozvaha možných řešení – Frameworky

Při návrhu webového rozhraní pro systém chytré domácnosti bylo nutné zvolit vhodné technologie. Klíčovými kritérii při rozhodování byly nízká hardwarová náročnost vzhledem k nasazení na Raspberry Pi, rychlost vývoje, dostupnost relevantních knihoven a celková složitost řešení.

2.1.1 Čistý přístup bez frameworku

Nejzákladnější možností bylo implementovat webový server přímo pomocí standardních knihoven zvoleného jazyka. Tento přístup nabízí maximální kontrolu nad chováním aplikace a nulové externí závislosti. Nevýhodou je však nutnost ruční implementace veškeré logiky – routování URL, parsování požadavků, správy sessions i generování odpovědí, což výrazně prodlužuje dobu vývoje a zvyšuje pravděpodobnost chyb.

2.1.2 Node.js – Express.js

Express.js je minimalistický framework pro prostředí Node.js, který je díky asynchronní povaze JavaScriptu velmi výkonný při obsluze velkého počtu souběžných požadavků. Je široce rozšířený, dobře zdokumentovaný a disponuje rozsáhlým ekosystémem balíčků přes npm. Pro projekt chytré domácnosti by byl funkčně dostačující, avšak jeho použití by znamenalo provozovat dva oddělené jazykové ekosystémy, JavaScript na straně serveru a Python pro systémové skripty a senzorovou logiku, což komplikuje údržbu.

2.1.3 Node.js – Next.js

Next.js je moderní framework postavený nad Reactem, zaměřený na tvorbu full-stack webových aplikací se server-side renderingem a file-based routováním. Jeho silnou stránkou je vývoj komplexních uživatelských rozhraní s bohatou interaktivitou. Pro účely tohoto projektu však představuje výraznou komplikaci v rámci vývoje, přináší velkou složitost buildovacího procesu, vysoké nároky na paměť při běhu.

2.1.4 Java – Spring Boot

Spring Boot je průmyslovým standardem pro tvorbu robustních, škálovatelných webových aplikací v Javě. Nabízí komplexní ekosystém zahrnující dependency injection, vestavěnou podporu REST API, ORM přes JPA/Hibernate a rozsáhlé možnosti konfigurace. Tyto vlastnosti jsou však zároveň jeho nevýhodou v kontextu tohoto projektu, jelikož JVM má vysoké nároky na operační paměť a čas inicializace aplikace, což je v prostředí embedded zařízení jako Raspberry Pi zásadní omezení. Spring Boot je vhodný pro enterprise aplikace, nikoli pro lehké IoT systémy.

2.1.5 Go – Gin

Gin je rychlý a paměťově velmi úsporný webový framework pro jazyk Go, který díky kompilované povaze jazyka dosahuje výjimečného výkonu i na slabším hardwaru. Pro nasazení na Raspberry Pi by byl z hlediska výkonu pravděpodobně nejlepší volbou. Překážkou by však bylo opuštění Pythonu, protože Python je nejjednodušší pro komunikaci embedded zařízení mezi sebou.

2.1.6 Python – Django

Django je plnohodnotný webový framework pro Python, protože obsahuje vestavěný ORM, administrační rozhraní, autentizaci, systém migrací databáze i šablonovací engine. Pro rozsáhlé projekty je Django vhodnou volbou. V kontextu tohoto projektu však jeho striktní struktura a vysoká míra abstrakce přinášejí více komplikací než užitku. Konfigurace projektu, správa aplikací a dodržování konvencí Djanga by zabraly neúměrně mnoho času vzhledem k relativně jednoduché funkčnosti, kterou projekt vyžaduje.

2.1.7 Python – FastAPI

FastAPI je moderní, výkonný framework pro Python zaměřený na tvorbu REST API. Jeho silnými stránkami jsou automatická generace interaktivní API dokumentace (Swagger UI), striktní typová validace vstupů pomocí Pydantic a asynchronní zpracování požadavků. Byl by vhodnou volbou pro projekty, kde je API hlavním produktem. V tomto projektu však není rozsáhlé ani veřejné API prioritou, takže výhody FastAPI by zůstaly z velké části nevyužity.

2.1.7 Zvolené řešení – Python – Flask

Po zvážení všech alternativ byl jako výsledné řešení zvolen **Flask** – lehký mikroframework pro Python. Flask nestanovuje přísnou strukturu projektu, nepřináší zbytečné závislosti a zároveň pokrývá veškeré potřeby projektu. Tento framework jsem vybral také na základě osobních zkušeností, jelikož umožňuje všechny zásadní požadavky pro tvorbu API jako jsou například autentizace, limitování dotazů, modularita. Zásadní výhodou je velice strmá učící křivka, která zajistí snazší a rychlejší vývoj. Pomocí tohoto frameworku je možné vytvořit například API, ale disponuje i funkcemi pro renderování HTML stránek, což jsem nakonec využil.

2.2 Rozvaha možných řešení – Hardware

Původní myšlenkou bylo využít embedded zařízení Radxa, které není tolik populární jako některé jeho alternativy, ale má jednu zásadní výhodu, což je přímo zabudovaná flash paměť, která ovšem způsobuje veliký finanční rozdíl v řádech několika stovek, což bylo vzhledem ke koncepci zadání nepřijatelné. Existují sice modely Radxa bez flash paměti, ale ty zase nemají například WiFi anténu, nebo Bluetooth. Proto jsem byl nucen hledat alternativy, kterých není málo, ale mezi nejpopulárnější a ověřené řešení patří Raspberry a OrangePi. Mezi těmito zařízeními není žádný zásadnější rozdíl, pokud porovnáváme rovnocenné modely. V konečném důsledku jsem zvolil Raspberry kvůli jeho dostupnosti a zkušenostem s touto značkou.

2.3 Návrh

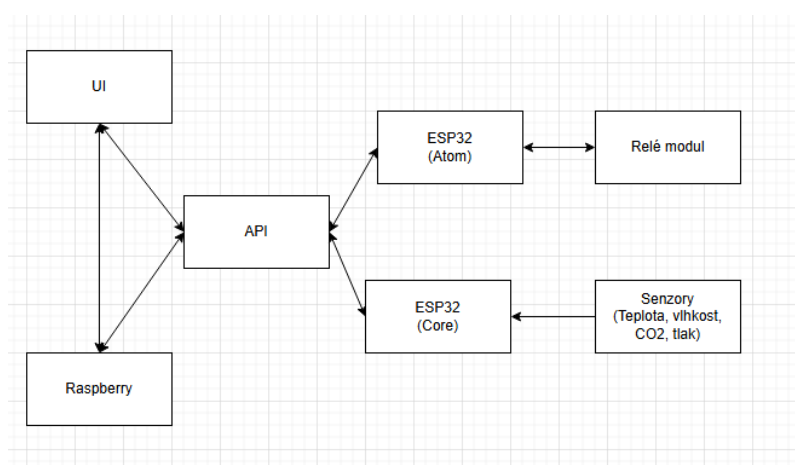
Po provedení počáteční rozvahy jsem se rozhodl pro programovací jazyk Python a framework Flask, které přesně sedí na koncept této práce, což je „lightweight“ aplikace pro chytrou domácnost, běžící na embedded zařízení.

Jako hardware jsem zvolil Raspberry na němž běží linux distribuce Armbian. Linux distribucí je obrovské množství, ale pro Raspberry se velmi často používá Raspberry OS, které dělá z Raspberry spíše desktop než embedded zařízení.

V zadání byla dále zmíněna simulace dat po ethernetu, což bylo nahrazeno reálnými daty z mé domácí „meteostanice“. Data jsou ze dvou ESP32 (M5stack), které mohou být napájeny z baterie nebo ze sítě. Jedná se o low power zařízení často používané pro prototypování.

Databáze byla použita SQLite, která jako jedna z mála může běžet na lokálním zařízení. Díky tomu zůstává toto řešení nezávislé na internetu, což byla hlavní priorita tohoto zadání.

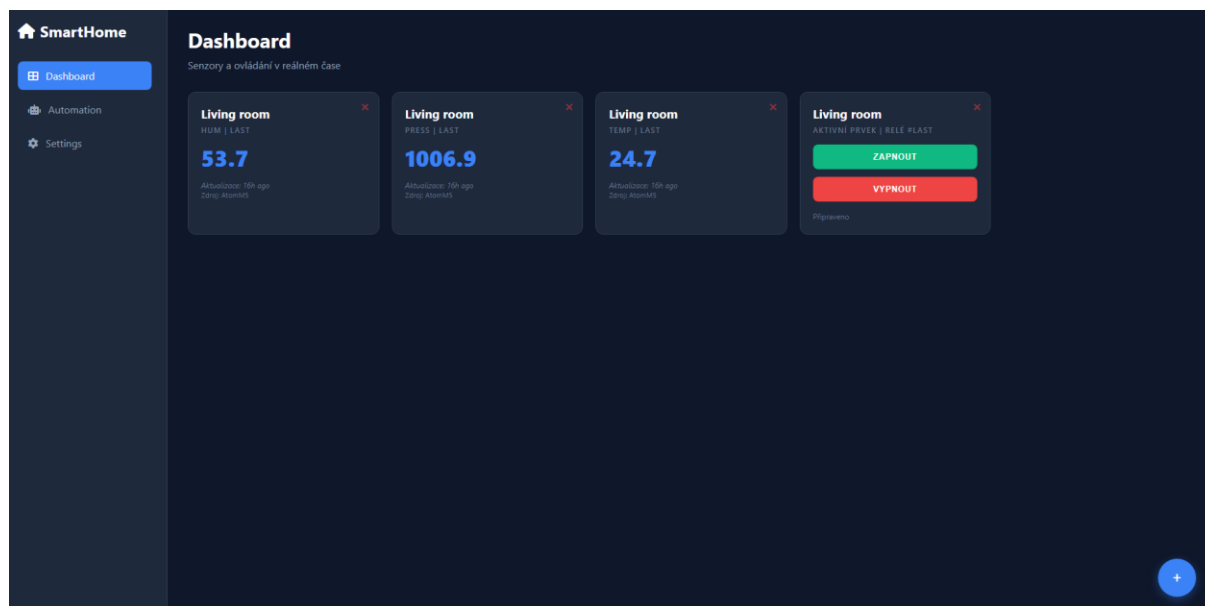
Protože by použití standardních IoT technologií (LoRa, ZigBee apod.) bylo drahé, tak je komunikace realizována pomocí standardních http requestů, což má jeden zásadní limit a tím je požadavek na stejnou síť všech zúčastněných zařízení. Tento požadavek byl vyřešen díky vytvoření sítě na Raspberry a na tuto síť se připojují ESP32 a posílají data. Samozřejmě jde dosáhnout stejného výsledku v případě, že jsou všechna zúčastněná zařízení připojená na stejnou síť (např. domácí síť). Na obrázku je vidět původní teoretický návrh projektu, který se podařilo naplnit. **Obrázek 3**



Obrázek 3: Návrh projektu

2.4 Realizace aplikace

Aplikace byla realizována v prostředí Pycharm Professional s využitím remote deploymentu, aby se aplikace vždy spouštěla na finálním zařízení, tedy Raspberry. Vývoj byl prováděn v iterativních způsobem, kdy v jednotlivých krocích docházelo k přidávání nových funkcionalit. Tento text popisuje strukturu aplikace, komunikaci s databází, obsluhu api a UI. Tento přístup umožnil průběžné testování jednotlivých částí aplikace a jejich úpravu dle zpětné vazby a měnících se požadavků. Obrázek znázorňuje webové rozhraní aplikace. **Obrázek 4**



Obrázek 4: Ukázka UI

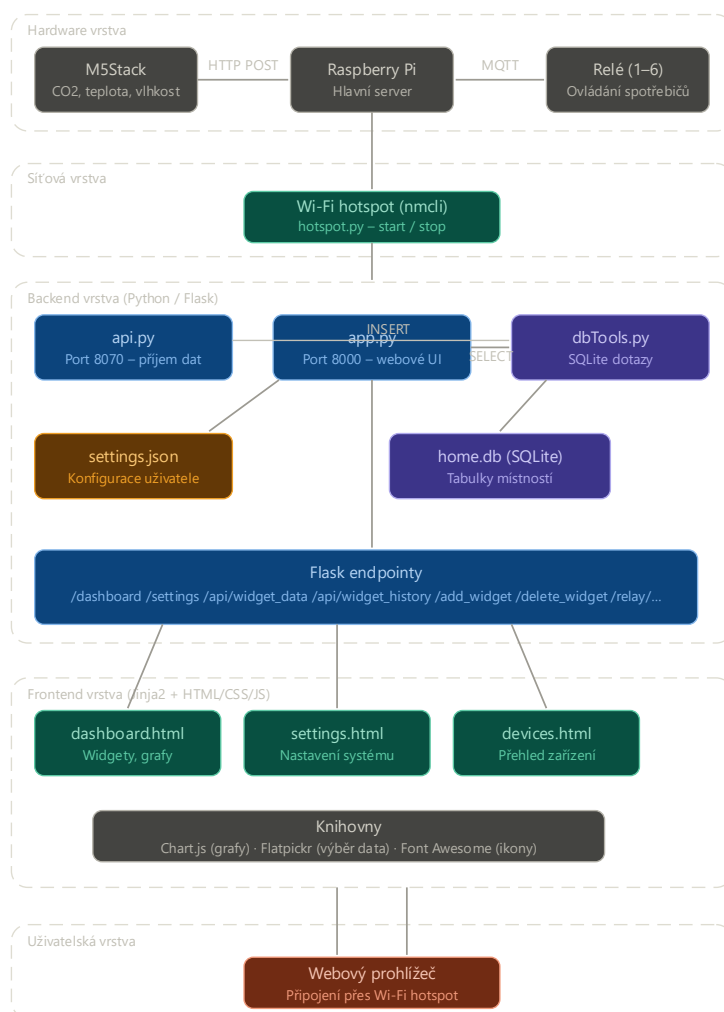
2.4.1 Zabezpečení aplikace

V rámci zabezpečení API byly provedeny úpravy výhradně na straně serveru, bez nutnosti změny firmware senzorových uzlů ESP32/M5Stack. Konkrétně byla opravena zranitelnost SQL Injection přidáním validace názvu tabulky pomocí regulárního výrazu, který povoluje pouze písmena, čísla a podtržítka. Na všechny odpovědi serveru byly přidány bezpečnostní HTTP hlavičky X-Content-Type-Options, X-Frame-Options a X-XSS-Protection. Byl také implementován jednoduchý rate limiting omezující počet požadavků na 60 za minutu pro jednu IP adresu, čímž se snižuje riziko útoku hrubou silou. Autentizace uživatelského rozhraní pomocí session a omezení CORS byly záměrně vynechány, neboť jejich implementace by vyžadovala úpravy na straně klientských zařízení a další časovou náročnost. Aplikace by měla sloužit primárně jako proof of concept pro budoucí bakalářskou práci.

2.5 Funkcionalita

Aplikace tvoří komplexní webový systém pro monitoring a ovládání chytré domácnosti, provozovaný lokálně na zařízení Raspberry Pi bez závislosti na internetu nebo cloudových službách. Systém se skládá ze dvou Flask serverů, jelikož jeden tvoří backend a druhý frontend. Jeden slouží k obsluze webové stránky a jejich renderování a druhý je pro obsluhu databáze. Na obrázku je vidět schéma aplikace.

Obrázek 5



Obrázek 5: Návrh aplikace

2.5.1 Sběr a ukládání dat ze senzorů

Senzorové uzly M5Stack(Atom a Core) průběžně měří hodnoty teploty, vlhkosti a koncentrace CO2 v jednotlivých místnostech. Naměřená data jsou odesílána formou HTTP POST požadavků na endpoint /data serveru api.py běžícího na portu 8070. Každý požadavek obsahuje v hlavičce identifikátor místnosti, podle kterého server automaticky vytvoří odpovídající tabulku v SQLite databázi home.db, pokud ještě neexistuje, a nový záznam do ní zapíše včetně časové značky. Díky tomuto přístupu je systém snadno rozšiřitelný o další senzorové uzly bez nutnosti jakékoli úpravy kódu.

2.5.2 Webový dashboard a widgety

Hlavní uživatelské rozhraní je přístupné přes webový prohlížeč na portu 8000 a tvoří ho dynamický dashboard s konfigurovatelnou sadou widgetů. Uživatel může přidávat nové widgety prostřednictvím modálního dialogu, kde zvolí název, zdrojovou místnost (tabulku v databázi), typ senzoru a způsob výpočtu zobrazované hodnoty – aktuální hodnotu, průměr, minimum nebo maximum. Každý widget zobrazuje živou hodnotu automaticky obnovovanou v nastavitelném intervalu a časový údaj o poslední aktualizaci. Data jsou načítána asynchronně přes endpoint `/api/widget_data` bez nutnosti znovunačtení stránky. Widgety lze rovněž odstraňovat přímo z dashboardu.

2.5.3 Historická data a grafické zobrazení

Kliknutím na libovolný senzorový widget se zobrazí překryvný panel s interaktivním grafem historie naměřených hodnot vykresleným pomocí knihovny Chart.js. Uživatel může zobrazit data za předdefinovaná časová období – týden, měsíc nebo rok – nebo zadat vlastní rozsah pomocí přesného výběru data a času prostřednictvím knihovny Flatpickr. Historická data jsou načítána z endpointu `/api/widget_history` a graf se automaticky přizpůsobí množství dat – při velkém počtu záznamů jsou skryty datové body pro lepší přehlednost.

2.5.4 Ovládání relé a fyzických spotřebičů

Systém umožňuje nejen pasivní monitoring, ale také aktivní ovládání připojených spotřebičů prostřednictvím relé modulů. Pro každé ze šesti dostupných relé lze přidat dedikovaný widget přímo na dashboard, který obsahuje tlačítka pro zapnutí a vypnutí. Po stisknutí tlačítka je odeslán HTTP POST požadavek na endpoint `/relay/{id}/{akce}` serveru `api.py`, který přeloží příkaz do MQTT zprávy a odešle ji přes lokálního MQTT brokera na příslušné relé. Stav odeslání je uživateli okamžitě zobrazen přímo ve widgetu.

2.5.5 Nastavení systému

Aplikace disponuje stránkou nastavení, kde může uživatel konfigurovat základní parametry systému – jméno správce, notifikační e-mail, interval automatické aktualizace dat, jednotky zobrazování teploty (Celsius nebo Fahrenheit) a aktivaci nočního bezpečnostního režimu. Veškerá nastavení jsou ukládána do souboru `settings.json` přes endpoint `/api/save_settings` a při každém spuštění aplikace automaticky načítána, přičemž pro případ chybějícího nebo poškozeného souboru jsou definovány výchozí hodnoty.

2.5.6 Síťová infrastruktura

Při spuštění aplikace Raspberry Pi automaticky vytvoří Wi-Fi hotspot pomocí nástroje nmcli prostřednictvím funkcí v souboru hotspot.py. Uživatelé i senzorové uzly M5Stack se připojují k této lokální síti, čímž celý systém funguje zcela samostatně a nevyžaduje existující domácí síťovou infrastrukturu ani přístup k internetu. Toto řešení zároveň zvyšuje bezpečnost systému, jelikož veškerá komunikace zůstává izolována v lokální síti.

Aplikaci je možné používat i v režimu lokální sítě, jelikož je potřeba dosáhnout stavu, aby naše aplikace byla ve stejném IP rozsahu jako zařízení, která posílají data. Přijdeme tím o izolovanost od okolního světa.

2.6 Testování

Testování aplikace probíhalo na dvou úrovních – automatizovaně pomocí jednotkových a integračních testů a manuálně ověřením funkčnosti na reálném hardwaru.

2.6.1 Automatizované testování

Pro automatizované testování byl zvolen framework pytest, který umožňuje psát přehledné testy jako prosté funkce s využitím systému fixtures pro přípravu testovacího prostředí. Testy jsou rozděleny do pěti souborů podle testovaného modulu – test_dbTools.py, test_hotspot.py, test_automation.py, test_api.py a test_app.py.

Jednotkové testy ověřují správnost izolovaných funkcí bez závislosti na reálném hardwaru. Modul dbTools.py je testován oproti skutečné SQLite databázi naplněné testovacími daty, přičemž se ověřují všechny výpočetní režimy – aktuální hodnota, minimum, maximum i průměr, včetně filtrování dat podle časového rozsahu. Logika hotspot.py je testována s mockem místo reálného systémového nástroje nmcli, čímž se ověřuje správnost předávaných parametrů bez nutnosti fyzicky měnit síťovou konfiguraci. Automation worker je testován s mockovanými hodnotami senzorů, přičemž se ověřuje správnost sepnutí a rozepnutí relé při překročení nastavených prahových hodnot, idempotentnost při opakovaném vyhodnocení stejného stavu a odolnost vůči výjimkám.

Integrační testy v souborech test_api.py a test_app.py využívají Flask testovací klient, který simuluje reálné HTTP požadavky bez nutnosti spouštět server. Testují se všechny endpointy obou Flask aplikací, přidávání a mazání widgetů, načítání historických dat, ovládání relé, ukládání nastavení i správa automatizačních pravidel.

```

import os
import sys
import sqlite3
import pytest
from unittest.mock import patch

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from src.tools import dbTools

DB = '/tmp/test_dbtools.db'

@pytest.fixture(autouse=True)
def fresh_db():
    con = sqlite3.connect(DB)
    con.execute("DROP TABLE IF EXISTS room")
    con.execute("CREATE TABLE room (CO2 FLOAT, Temp FLOAT, Hum FLOAT, Tim TEXT)")
    con.executemany("INSERT INTO room VALUES (?, ?, ?, ?)", [
        (400, 20.0, 50.0, '2026-01-01 10:00:00'),
        (500, 22.5, 55.0, '2026-01-02 10:00:00'),
        (600, 25.0, 60.0, '2026-01-03 10:00:00'),
    ])
    con.commit(); con.close()
    yield
    if os.path.exists(DB):
        os.remove(DB)

@pytest.fixture
def db():
    return patch('dbTools.sqlite3.connect', return_value=sqlite3.connect(DB))

# — get_db_tables —————

def test_get_db_tables_returns_list(db):
    with db: assert isinstance(dbTools.get_db_tables(), list)

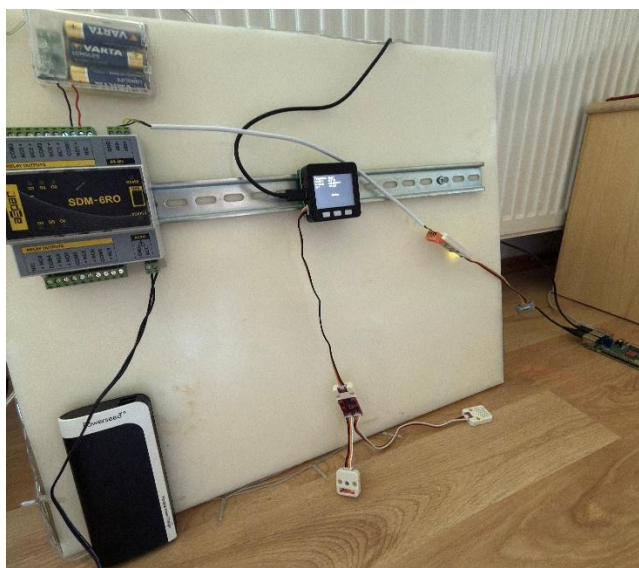
def test_get_db_tables_contains_table(db):
    with db: assert 'room' in dbTools.get_db_tables()

```

Zdrojový kód 1.1: Ukázka testování modulu db_tools.py

2.7.1 Manuální testování

Část testování probíhala manuálně přímo na cílovém hardwaru Raspberry Pi s připojenými senzory M5Stack a relé moduly. Po spuštění aplikace bylo ověřeno, že Raspberry Pi úspěšně vytvoří Wi-Fi hotspot a webové rozhraní je dostupné z mobilního telefonu i notebooku připojených ke stejné síti, popřípadě k síti vytvořené pomocí Raspberry. Na dashboardu byly přidány widgety pro jednotlivé místnosti a senzory a bylo ověřeno, že se hodnoty správně načítají z databáze a aktualizují v nastaveném intervalu. Funkčnost grafů historických dat byla ověřena výběrem různých časových rozsahů i vlastního datumového rozmezí. Ovládání relé bylo otestováno fyzickým připojením spotřebiče a ověřením, že stisk tlačítka na dashboardu skutečně sepne nebo rozepne obvod. Automatizační pravidla byla otestována simulací podmínek – zahřátím senzoru nad nastavený práh a kontrolou, že se relé skutečně sepne a po ochlazení opět rozepne. Nastavení aplikace bylo uloženo a ověřeno, že přetrvá i po restartu serveru. V rámci testování bylo testováno chování po výpadku elektřiny, které by nemělo aplikaci nijak ovlivnit, jelikož všechna potřebná data jsou ukládána do persistentního úložiště. Testování probíhalo na mnou vytvořené desce, která slouží k demonstraci funkčnosti stránky, kde budou všechna použitá zařízení. **Obrázek 6**



Obrázek 6: Ukázka testování

Každá karta je rozkliknutelná a po jejím rozkliknutí se objeví graf, který vyobrazuje danou hodnotu v čase definovaném uživatelem. Na obrázku jsou vidět hodnoty mého pokoje, ovlivněné automatickým větráním v kombinaci s orientací ke slunci. Proto se zde vyskytují takové výkyvy.

Obrázek 7



Obrázek 7: Ukázka naměřených hodnot

Závěr

Cílem této práce bylo navrhnout a realizovat webovou aplikaci pro správu chytré domácnosti běžící na jednodeskovém počítači Raspberry Pi. Vývoj probíhal s jasně definovaným zadáním a praktickým zaměřením, přičemž klíčovým aspektem bylo vytvoření funkčního systému umožňujícího sběr, ukládání a vizualizaci dat ze senzorů a zároveň ovládání připojených spotřebičů prostřednictvím webového rozhraní.

V průběhu návrhu bylo nejprve provedeno srovnání dostupných frameworků a technologií pro tvorbu webových aplikací. Po zvážení požadavků projektu, hardwarových omezení Raspberry Pi a vlastních zkušeností byl jako nejvhodnější řešení zvolen framework Flask, který nabízí dostatečnou funkcionalitu při minimální režii a nízké paměťové náročnosti. Aplikace byla rozdělena do dvou samostatných Flask serverů – jeden zajišťuje webové rozhraní a druhý příjem dat ze senzorů a ovládání relé prostřednictvím protokolu MQTT.

V průběhu vývoje bylo nutné řešit řadu praktických problémů, například zajištění spolehlivé komunikace mezi senzory M5Stack a serverem, správnou synchronizaci automatizačního workeru běžícího na pozadí nebo stabilitu Wi-Fi hotspotu spravovaného přímo Raspberry Pi. Díky iterativnímu přístupu k vývoji se podařilo postupně všechny tyto problémy vyřešit a aplikaci stabilizovat do funkčního celku.

Výsledná aplikace umožňuje sběr a ukládání naměřených hodnot teploty, vlhkosti a koncentrace CO₂ do SQLite databáze, jejich statistické zpracování a vizualizaci formou interaktivních grafů s možností filtrování podle časového rozsahu. Uživatel může prostřednictvím dashboardu přidávat a konfigurovat widgety pro jednotlivé místnosti a senzory, ovládat až šest relé modulů a definovat automatizační pravidla, která na základě naměřených hodnot automaticky sepnou nebo rozepnou připojený spotřebič. Testování prokázalo, že aplikace je funkční a spolehlivá v podmínkách lokální sítě vytvořené přímo Raspberry Pi.

Navržený systém v aktuální podobě plně pokrývá požadavky zadání a je vhodný pro nasazení v domácím prostředí jako autonomní lokální řešení bez závislosti na internetu nebo cloudových službách. Určitým omezením zůstává skutečnost, že widgety a automatizační pravidla jsou ukládána pouze v operační paměti a po restartu serveru se ztratí. Dalším omezením je chybějící autentizace, která by v produkčním nasazení byla žádoucí. Výsledky práce přesto ukazují, že jednodeskové počítače typu Raspberry Pi jsou plně schopné provozovat komplexní webové aplikace pro správu chytré domácnosti. Další rozvoj by se mohl zaměřit na persistentní ukládání konfigurace widgetů do databáze, přidání uživatelské autentizace, podporu více uživatelů nebo rozšíření o notifikace prostřednictvím e-mailu při překročení nastavených prahových hodnot senzorů.

Bibliografie

1. **Zenkl, Petr.** Prezentace k předmětu Telekomunikační technologie. místo neznámé : Nezveřejněno, 2023.
2. **Radxa.** Radxa Official Documentation. [Online] <https://docs.radxa.com/>.
3. **Raspberry Pi Foundation.** Raspberry Pi Documentation. [Online] <https://www.raspberrypi.com/documentation/>.
4. **Orange Pi.** Orange Pi Official Website. [Online] <http://www.orangepi.org/>.
5. **Ronacher, Armin.** Flask Documentation (3.x). [Online] 2010–2025. <https://flask.palletsprojects.com/>.
6. **Hypertext Transfer Protocol – HTTP.** *GeeksforGeeks*. [Online] [Citace: 12. 5 2026.] <https://www.geeksforgeeks.org/html/what-is-http/>.
7. **Web framework.** *Wikipedia: the free encyclopedia*. [Online] [Citace: 12. 5 2026.] https://en.wikipedia.org/wiki/Web_framework.
8. **M5Stack.** M5Stack Official Documentation. [Online] [Citace: 12. 5 2026.] <https://docs.m5stack.com/>.
9. **Hrnčíř, Pavel.** Prezentace k předmětu automatizace. 2023.
10. **Inc, Tailscale.** How Tailscale Works. [Online] <https://tailscale.com/blog/how-tailscale-works/>.
11. **Richard, D.** SQLite Documentation. [Online] <https://www.sqlite.org/docs.html>.

Přílohy

A Příloha

- [Zdrojové kódy](#)

A.1 Zdrojové kódy

Kompletní zdrojové kódy použité v této práci jsou veřejně dostupné v online repozitáři na GitHubu:
<https://github.com/Martinos21/HomeApp?tab=readme-ov-file>